

Tackling Re-buffering in Adaptive Video Streaming over Dynamic Networks: A Generative AI Approach

Duc V. Nguyen

*Department of Information and Communication Engineering
Tohoku Institute of Technology
Sendai, Japan
ducnguyen@tohotech.ac.jp*

Abstract—Re-buffering is one of the most common factors that reduce the user’s Quality of Experience in online streaming video. In this paper, we introduce a novel solution to tackle the re-buffering problem in video streaming using generative Artificial Intelligence (genAI). In our proposed solution, a generative AI model is trained and executed to generate video frames in case of sudden network bandwidth drops. For that purpose, we first formulate the genAI-assisted adaptation problem in adaptive video streaming as an optimization problem. We then present a lightweight adaptation algorithm that effectively reduces the number of re-buffering events under challenging network conditions.

Index Terms—Adaptive Video Streaming, Re-buffering, Generative AI

I. INTRODUCTION

Video streaming is one of the most important Internet applications, accounting for more than 65% of the total Internet traffic [1]. To deal with dynamic network conditions and heterogeneous user devices, adaptive bitrate streaming (ABR) technology, such as MPEG-DASH, has been proposed and has become the de facto standard for Internet video streaming. In ABR, the original video is divided into multiple chunks called *segment*, and each is encoded into various quality levels. The streaming client running on the user’s device selects the most suitable quality version for each video segment to maximize the user’s Quality of Experience [2].

Re-buffering is one of the primary problems in video streaming over dynamic networks. Re-buffering occurs when one or more requested video segments do not arrive at the user device before their playback deadlines, resulting in a temporary stop in video playback. The main cause of re-buffering is usually a significant reduction in available network bandwidth due to factors such as user mobility, bad weather, and out-of-sight in wireless networks [3]. Re-buffering causes a significant reduction in the user’s viewing experience [4]. To reduce the number of re-buffering, existing adaptive streaming solutions prefetch and store several video segments in a buffer in advance. Generally, a large buffer size is better for coping with network throughput fluctuations. However, small buffer sizes are demanded in many cases, such as live streaming and short video streaming [5].

In the field of AI research, advanced generative AI is revolutionizing video creation by allowing models (e.g., OpenAI’s Sora [6]) to generate realistic video content from various types

of input, including text, images, or audio. So far, running advanced AI models on consumer devices is difficult due to the constrained resources. However, AI chips are being integrated into consumer devices, enabling on-device AI capabilities [7].

In this paper, we present a novel AI-assisted solution to tackle the re-buffering problem in adaptive video streaming over dynamic networks with small buffer sizes. In our solution, when a sudden degradation in available network bandwidth is detected, a generative AI model is executed on-device to generate video frames, which are played back in case the requested video chunk has not arrived on time. If the requested video segment does not arrive before its playback deadline, then the generated video frames will be played to ensure the smoothness of the streaming session.

The remainder of the paper is as follows. Related works are described in Section II. The proposed AI-assisted streaming framework is presented in Section III, followed by an evaluation in Section IV. Section V concludes the paper and discusses directions for future work.

II. RELATED WORK

Artificial Intelligence (AI) is being integrated across the video streaming industry to enhance both the service’s reliability and the user’s quality of experience. For content preparation, AI is used to optimize traditional video codecs, such as using neural networks to improve the accuracy of motion vector prediction by replacing traditional block-based compression with end-to-end neural networks [8]. AI-based video codecs such as Facebook’s Deep Video Compression [9] achieve high compression with better perceptual quality at lower bitrates compared to traditional codecs. To optimize video transmission over networks, AI-based adaptive bitrate (ABR) algorithms such as Pensieve [10] have been proposed and shown to achieve state-of-the-art performance. In AI-based solutions, reinforcement learning is mainly used to train quality selection agents that maximize the reward in the form of the user’s Quality of Experience. In addition, AI techniques such as super-resolution [11] are used to improve the video quality on the end-user’s device.

Adaptive solutions for low-latency video streaming have been proposed, including both AI-based and heuristic solutions. Heuristic solutions such as Stallion [12] use a throughput-based adaptation strategy, considering both the

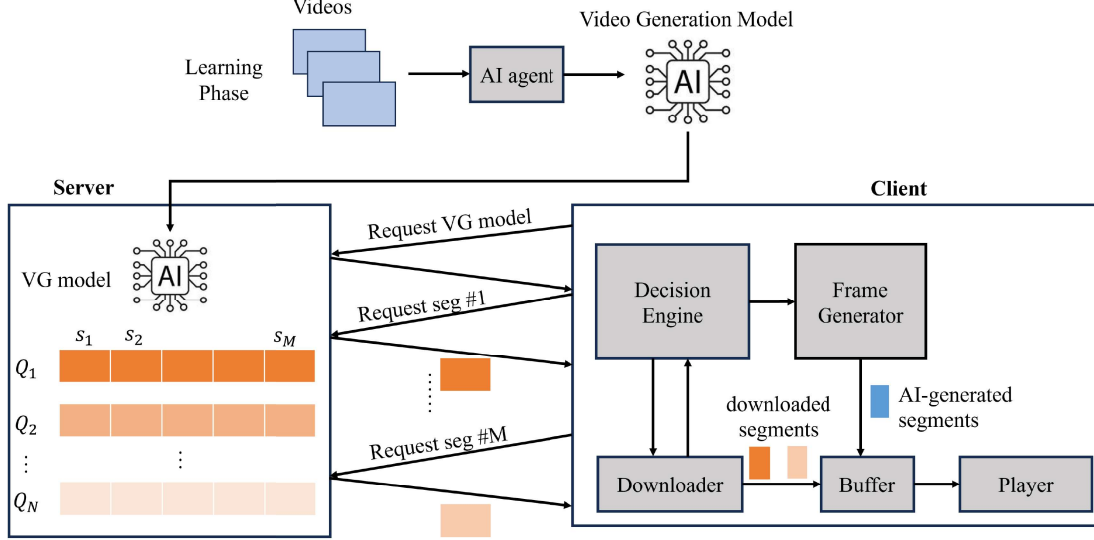


Fig. 1: General System Architecture of the Proposed System

mean and standard deviation of available network bandwidth. AI-based solutions such as Learn2Adapt [13] use historical feedback to make real-time decisions, achieving sub-2-second latency with high QoE. Previous AI-based solutions mainly use reinforcement learning to replace heuristic rules in traditional methods. However, the use of generative AI in tackling re-buffering has not been investigated.

III. PROPOSED FRAMEWORK

The general architecture of the proposed framework is shown in Fig. 1, which consists of a streaming server and a streaming client connected over a highly dynamic network such as a mobile network. In the learning phase, pre-recorded videos are used to train a video frame generation model. The trained model is then stored on the streaming server in advance. At the beginning of the streaming session, the streaming client downloads the video frame generation model from the server, and performs the necessary pre-processing to reconstruct the model at the client. Multiple versions of the original video divided into segments are stored on the server.

The streaming client consists of a downloader, a decision engine, a frame generator, a buffer, and a player. The downloader is responsible for sending requests and downloading video segments from the server based on the decision made by the decision engine. The decision engine also decides whether to generate a video frame based on the network conditions, and the current buffer level. The frame generator will generate video frames once it receives a request from the decision engine. The AI-generated segments will be placed in the client buffer.

A. Problem Formulation

Assume that the system need to stream a video from a server to a streaming client over a dynamic network. On the server, the video is divided into M τ -second segments, each

is encoded into N versions. Let $R(i)$ denotes the bitrate of version i ($1 \leq i \leq N$) where $R(i) < R(i+1)$. The client downloads the video segments from the server in a sequential order. To deal with varying network condition, the client stores downloaded video segments in a buffer of size B_s before starts playing.

At a time t_m^r , the client decides a quality level $q_m \in \{1, 2, \dots, N\}$ of segment m ($1 \leq m \leq M$) and sends a request to the server to download the chosen segment. Let BW_m denote the average network bandwidth, the time t_m^d required to download segment m is given by,

$$t_m^d = R(q_m) \times \tau / BW_m \quad (1)$$

Let $b(t)$ denote the buffer level at time t , a *re-buffering event* occurs if the segment download time t_m^d is higher than the buffer level at the beginning of the download, i.e., $t_m^d > b(t_m^r)$. Generally, the decision engine will estimate the available bandwidth and select a version whose bitrate is smaller than the estimated bandwidth value to ensure that the segment will arrive before its playback deadline. However, the network bandwidth might be significantly lower than the estimated value during the download period of segment m .

In this work, we propose to use generative AI to reduce the probability of re-buffering in case of significant network bandwidth degradation. In particular, the client continuously monitors the network condition. If the client detects a significant change in available network bandwidth that might lead to re-buffering, the client will generate video frames using the pre-trained generative AI model. The generated video frames will be played in case the requested segment does not arrive on time. Assume that the client can generate video frames using a pre-trained generative AI model $h_{\Theta}(\cdot)$ that is capable of generating F_g frames per second, given a set of input video frames X_m . Let $t_m^g > t_m^r$ be the time client starts generating video frames, F be the frame rate of the original video, and

the duration of the generated video chunk τ_k^g before the client buffer depletes is given by,

$$\tau_k^g = \frac{(b(t_m^r) - (t_m^g - t_m^r)) \times F_g}{F} \quad (2)$$

Let u_k be a decision variable indicating whether the AI-based video generation is performed, i.e.,

$$u_m = \begin{cases} 1, & \text{if AI-based video generation is performed} \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

, the re-buffering duration $Re(m)$ at segment m is given by,

$$Re(m) = \max(t_m^d - u_m \times \tau_m^g - b(t_m^r), 0) \quad (4)$$

AI-based video generation is a resource-intensive task as it usually requires using Graphical Processing Unit (GPU). Let C_m denote the cost associated with performing AI-based video generation of segment m , the generative AI-assisted video adaptation problem can be stated as follows.

Given the chosen quality q_m , the segment request time t_m^r , and the video generation model $h_\Theta(\cdot)$, decide the values of $\{u_m, t_m^g, X_m\}$ for each segment m ($1 \leq m \leq M$) to minimize the total re-buffering time during the streaming session:

$$\text{minimize } \sum_{m=1}^M Re(m) \quad (5)$$

subject to:

$$\sum_{m=1}^M C_m \leq C^{\max} \quad (6)$$

In Eq. (6), $C^{\max}$ denotes the maximum allowable cost of performing AI-based video generation. This value depends on various factors such as the capacity of the user device and the power consumption of the AI model. In this work, we assume that the user device has sufficient resources to support AI-based video generation. The task of modeling the cost C_m and solving the problem in the case of resource-constrained clients will be considered in our future work.

B. Video Generation Model

Generative AI solutions for video generation have been developed in the literature. In this paper, we employ the *SimVP* model proposed in [14] to perform this task due to its simplicity and high performance. Other models will be considered in our future work. Given an input of Ψ past video frames $X = \{x_1^p, x_2^p, \dots, x_\Psi^p\}$, the task is to learn a model $h_\Theta(\cdot)$ that generates Ω future frames $Y = \{x_1^f, x_2^f, \dots, x_\Omega^f\}$.

$$Y = h(X) \quad (7)$$

The *SimVP* model consists of three components, namely an encoder, a translator, and a decoder, all built on a Convolutional Neural Network (CNN). The encoder extracts spatial features from the input video frames. In particular, the encoder uses a stack of N_s 2D convolution blocks to extract the hidden feature z_i as follows.

$$z_i = \sigma(\text{LayerNorm}(\text{Conv2D}(z_{i-1}))), 1 \leq i \leq N_s \quad (8)$$

The task of the translator is to learn the temporal evolution among video frames using N_t Inception modules, which consists of a 2D convolutional layer with 1×1 kernel followed by parallel GroupConv2D operators.

$$z_j = \text{Inception}(z_{j-1}), N_s < j \leq N_s + N_t \quad (9)$$

The learned spatial and temporal features are then used by the decoder to generate future video frames using unConvNorm-ReLU blocks.

$$z_k = \sigma(\text{GroupNorm}(\text{unConv2d}(z_{k-1}))), \\ N_s + N_t < k \leq 2N_s + N_t$$

The model parameters are trained to minimize the mean squared loss (MSE).

$$\Theta^* = \text{argmin } \text{MSE}(h_\Theta(X), Y) \quad (10)$$

In this work, the number of output video frames Ω is set to the number of frames in a video segment, i.e., $\Omega = \tau \times F$.

C. Adaptation Algorithm

Given the video generation model described in Section III-B, we develop an adaptation algorithm that decides the values of $\{u_m, t_m^g, X_m\}$ for every segment m during a streaming session. To decide the value of u_m , the proposed algorithm continuously monitor the available network bandwidth during the download of segment m . In particular, let $TD(t)$ denote the total downloaded data of segment m at time t , a sample bandwidth $s(i)$ ($i \geq 1$) is calculated for every Δt seconds.

$$s(i) = \frac{TD(t_m^r + i \times \Delta t) - TD(t_m^r + (i-1) \times \Delta t)}{\Delta t} \quad (11)$$

The remaining download time of segment m is estimated using the latest bandwidth sample $s(i_{last})$ as follows.

$$t_m^{d_{remain}} = \frac{R(Q_m) \times \tau - TD(t_m^r + i_{last} \times \Delta t)}{s(i_{last})} \quad (12)$$

If the estimated remaining download time is higher than the remaining buffer size, then AI-based video generation will be performed immediately, i.e.,

$$u_m = \begin{cases} 1, & \text{if } t_m^{d_{remain}} > b(t_m^r) - i_{last} \times \Delta t \\ 0, & \text{otherwise} \end{cases} \quad (13)$$

$$t_m^g = t_m^r + i_{last} \times \Delta t \quad (14)$$

Let J denote the sequential number of the last displayed video frame. The input of the frame generation model X_m is given by:

$$X_m = \{x_{J-\Psi+1}, \dots, x_{J-1}, x_J\} \quad (15)$$

The client will continuously generate video frames until the download of segment m is completed. It should be noted that if AI-based video generation is not performed (i.e., $u_m = 0$), the algorithm simply waits until the download of the current segment complete, then requesting the next segment ($m+1$).

To decide the quality level q_m of segment m , we adopt a simple strategy based on the average network bandwidth

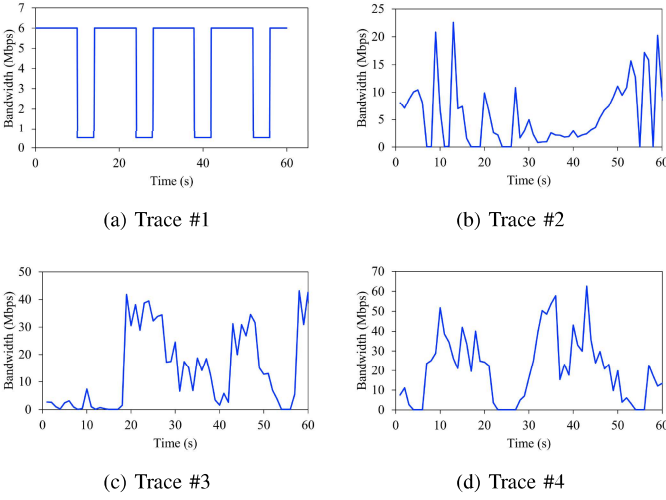


Fig. 2: Bandwidth network traces used in the experiment. Trace #1 is a synthesized trace, while the other traces are recorded in a mobile network.

during the download the last segment. Let t_{m-1}^d be the total download time of segment $(m-1)$, the average download bandwidth of segment $(m-1)$ is given by,

$$BW_{m-1} = R(q_{m-1}) \times \tau / t_{m-1}^d \quad (16)$$

The quality level q_m of segment m is chosen as the highest quality level whose bitrate is lower than BW_{m-1} as follows.

$$q_m = \max (i \mid R(i) \leq BW_{m-1}) \quad (17)$$

In addition, if the buffer size drops below a critical threshold B_{low} , the client will select the lowest quality level for the following segments until the buffer size achieves a maintenance level B_{main} .

IV. EVALUATION

We evaluate the proposed method with trace-driven simulation. The streaming server is implemented using an Apache Web server hosting video segments. For the video content, we use the BigBuckBunny video from the open dataset of [15]. The video duration is 60 seconds with a frame rate of $F=24$ fps. The segment duration τ is 1 second (i.e., $M=60$). The number of quality versions N is set to 5. The initial buffer size B_s is set to 4 seconds. The critical buffer size B_{low} and the maintenance buffer size B_{main} are set to 2 seconds and 4 seconds, respectively. Video segment download is implemented using the HTTP GET method. The server and the client are running on separate Ubuntu OS machines. The network bandwidth between the client and the server is emulated using a synthesized trace and three actual network traces, as shown in Fig. 2. The actual network traces are obtained from the dataset in [16]. The proposed method is compared to a reference method where the client waits until the requested segment is completely downloaded, and then sends a request for the next segment. The version of each

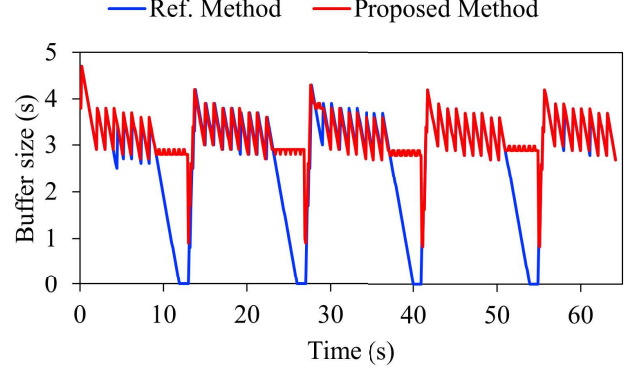


Fig. 3: Buffer size over time of the proposed and reference method under the network trace #1.

segment is chosen based on the download throughput of the previous segment.

In our experiment, the video generation model $h_{\Theta}(\cdot)$ is trained on the BigBuckBunny video with model parameters are set to those described in the original paper. The number of input video frames (Ψ) and output video frames (Ω) are set to 10 and 24, respectively. The original video is processed into seventy short videos, with sixty videos are used for training and the remaining ten videos used for testing. The model is trained on an NVIDIA GeForce RTX 3090 24 GB. The video generation framerate of the trained *simVP* model is $f^g = 24.8$ fps.

Fig. 3 shows the buffer size over time of the proposed and reference methods under the synthesized bandwidth trace (Fig. 2a), where the bandwidth drops from 6 Mbps to 0.5 Mbps every 10 seconds. It can be seen that when the network bandwidth drops significantly (e.g., $t=10$ s), the proposed method can maintain the buffer size and thus provide smooth playback. Meanwhile, the reference method results in rebuffering events as the client's buffer depletes due to the late arrival of the requested video segment.

Table I shows the performance in terms of average segment bitrate, number of rebufferings, and the total rebuffering time in seconds of the proposed and reference methods under four network bandwidth traces. For the proposed method, the amount of AI-generated video and the playback duration of the AI-generated part are also reported. It can be seen that our proposed method achieves a similar average segment bitrate as that of the reference method while significantly reducing rebuffering. In particular, the proposed method does not yield any re-buffering under all network traces. However, it can also be seen that the amount of generated video duration is much larger than the amount of played video content. For instance, in the case of network trace #2, the generated video duration is 19.1 seconds while the played duration is only 1.2 seconds. The video parts that have been generated but not played can be considered as waste. The large amount of wasted content is likely because the proposed algorithm reacts immediately

TABLE I: Performance of the reference method and the proposed method under four different bandwidth traces.

Network Trace	Ref. Method			Proposed Method				
	Average Bitrate (kbps)	Number of rebuffering	Total Rebuffering time (s)	Average Bitrate (kbps)	Number of rebuffering	Total Rebuffering time (s)	Generated video duration (s)	Played video duration(s)
Trace #1	3850	4	4.0	3866.7	0	0	16.6	4.0
Trace #2	3266.7	2	1.8	3233.3	0	0	19.1	1.2
Trace #3	4016.7	2	4.7	4083.3	0	0	14.8	5.0
Trace #4	4200	2	2.5	4266.7	0	0	9.0	2.8

whenever a significant reduction in available bandwidth is detected. This issue will be improved in our future work.

V. CONCLUSION AND FUTURE WORK

In this paper, we introduce a novel generative AI-based solution to tackle the re-buffering problem in adaptive video streaming over highly dynamic networks. Trace-driven experiment results show that the proposed solution can effectively reduce the re-buffering time compared to a conventional solution. In future work, the effects of AI-generated content parts on the user's perception and experience will be evaluated. Additionally, solutions to reduce the amount of wasted AI-generated video will be explored.

REFERENCES

- [1] Sandvine, "THE GLOBAL INTERNET PHENOMENA REPORT 2023," <https://www.applogiconetworks.com/press-releases/sandvines-2023-global-internet-phenomena-report-shows-24-jump-in-video-traffic-with-netflix-volume-overtaking-youtube>, Jan. 2023.
- [2] Iraj Sodagar, "The mpeg-dash standard for multimedia streaming over the internet," *IEEE multimedia*, vol. 18, no. 4, pp. 62–67, 2011.
- [3] Haoyuan Zhao, Hao Fang, Feng Wang, and Jiangchuan Liu, "Realtime multimedia services over starlink: A reality check," in *The 33rd Workshop on Network and Operating System Support for Digital Audio and Video*, Vancouver, BC, Canada, 2023, pp. 43–49.
- [4] Huyen T. T. Tran, Nam Pham Ngoc, Anh T. Pham, and Truong Cong Thang, "A multi-factor qoe model for adaptive streaming over mobile networks," in *2016 IEEE Globecom Workshops (GC Wkshps)*, Washington DC, US, Dec. 2016, pp. 1–6.
- [5] Nguyen Tien Phong, Truong Thu Huong, Pham Ngoc Nam, Truong Cong Thang, and Duc Nguyen, "Joint preloading and bitrate adaptation for short video streaming," *IEEE Access*, vol. 11, pp. 121064–121076, 2023.
- [6] Yixin Liu, Kai Zhang, Yuan Li, Zhiling Yan, Chujie Gao, Ruoxi Chen, Zhengqing Yuan, Yue Huang, Hanchi Sun, Jianfeng Gao, Lifang He, and Lichao Sun, "Sora: A review on background, technology, limitations, and opportunities of large vision models," 2024.
- [7] Xubin Wang, Zhiqing Tang, Jianxiong Guo, Tianhui Meng, Chenhao Wang, Tian Wang, and Weijia Jia, "Empowering edge intelligence: A comprehensive survey on on-device ai models," *ACM Comput. Surv.*, vol. 57, no. 9, Apr. 2025.
- [8] Alex Borges, Bruno Zatt, Marcelo Porto, and Guilherme Correa, "A systematic literature review on video transcoding acceleration: challenges, solutions, and trends," *Multimedia Tools and Applications*, vol. 83, no. 24, pp. 64079–64108, 2024.
- [9] Matthew Muckley, Jordan Juravsky, Daniel Severo, Mannat Singh, Quentin Duval, and Karen Ullrich, "Neuralcompression," <https://github.com/facebookresearch/NeuralCompression>, 2021.
- [10] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh, "Neural adaptive video streaming with pensieve," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, Los Angeles, CA, USA, 2017, p. 197–210.
- [11] Yinjie Zhang, Yuanxing Zhang, Yi Wu, Yu Tao, Kaigui Bian, Pan Zhou, Lingyang Song, and Hu Tuo, "Improving quality of experience by adaptive video streaming with super-resolution," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 1957–1966.
- [12] Craig Gutterman, Brayn Fridman, Trey Gilliland, Yusheng Hu, and Gil Zussman, "Stallion: video adaptation algorithm for low-latency video streaming," in *Proceedings of the 11th ACM Multimedia Systems Conference*, Istanbul, Turkey, 2020, p. 327–332.
- [13] Theo Karagkioulos, Rufael Mekuria, Dirk Griffioen, and Arjen Wagenaar, "Online learning for low-latency adaptive streaming," in *Proceedings of the 11th ACM Multimedia Systems Conference*, Istanbul, Turkey, 2020, p. 315–320.
- [14] Zhangyang Gao, Cheng Tan, Lirong Wu, and Stan Z Li, "Simvp: Simpler yet better video prediction," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 3170–3180.
- [15] Stefan Lederer, Christopher Müller, and Christian Timmerer, "Dynamic adaptive streaming over http dataset," in *Proceedings of the 3rd Multimedia Systems Conference*, Chapel Hill, North Carolina, 2012, pp. 89–94.
- [16] J. van der Hooft, S. Petrangeli, T. Wauters, R. Huysegems, P. Rondao Alface, T. Bostoen, and F. De Turck, "HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks," *IEEE Communications Letters*, vol. 20, no. 11, pp. 2177–2180, 2016.